Университет ИТМО

Практическая работа №4

Динамическое программирование

Вариант 17

Студент: Никита Лютый
Поток: 1.1
Номер в потоке: 17

Санкт-Петербург
2026

1 Цель работы

Освоить метод динамического программирования для решения многоэтапных задач оптимизации: составить математическую модель, определить этапы, состояния и управления, вывести рекуррентное соотношение Беллмана, выполнить ручные расчёты, восстановить оптимальные планы и проверить результат программно.

2 Постановка задачи

Для номера в потоке 17 номер варианта определяется как остаток от деления на 20:

$$17 \bmod 20 = 17.$$

В варианте 17 необходимо распределить 12 смен между тремя производственными линиями. Каждая линия должна получить хотя бы одну смену. Первая линия запускается только при выделении ей не менее 4 смен. Требуется максимизировать суммарный выпуск.

Исходные данные:

Смен	1	2	3	4	5	6	7	8	9	10	11
Линия 1	22	28	30	18	21	23	25	26	27	28	29
Линия 2	6	11	15	18	20	22	23	24	25	25	26
Линия 3	5	10	14	17	19	21	22	23	24	24	25

Значения первой линии для 1–3 смен в данной задаче недопустимы, так как по условию первая линия должна получить не менее 4 смен.

Пусть x_i — количество смен, выделенное линии i, а $w_i(x_i)$ — соответствующий выпуск. Тогда

$$\boxed{W = w_1(x_1) + w_2(x_2) + w_3(x_3) \to \max}$$

при ограничениях

$$\boxed{x_1 + x_2 + x_3 = 12},$$

$$\boxed{x_1 \geq 4, \qquad x_2 \geq 1, \qquad x_3 \geq 1},$$

$$x_1, x_2, x_3 \in \mathbb{Z}.$$

3 Краткая теория

В динамическом программировании исходная задача представляется как последовательность этапов. Перед каждым этапом система находится в некотором состоянии, затем выбирается управление, после чего система переходит в новое состояние.

Для данной задачи:

- этап i — распределение смен линии i;
- состояние S_i — количество нераспределённых смен перед этапом i;
- управление x_i — количество смен, выделяемое линии i;
- результат этапа — выпуск $w_i(x_i)$;
- переход — $S_{i+1} = S_i - x_i$.

Начальное состояние:

$$S_1 = 12.$$

После последнего этапа ресурс должен быть использован полностью:

$$S_4 = 0.$$

Принцип оптимальности Беллмана: оставшаяся часть оптимальной стратегии должна быть оптимальной для состояния, достигнутого после уже принятых решений.

Определим $V_i(S_i)$ как максимальный суммарный выпуск, который можно получить начиная с линии i, если перед ней осталось S_i смен. Если текущей линии выделить x_i смен, она даст $w_i(x_i)$, а последующим линиям останется $S_i - x_i$ смен. Поэтому

$$\boxed{V_i(S_i) = \max_{x_i \in U_i(S_i)} \left[w_i(x_i) + V_{i+1}(S_i - x_i) \right]}.$$

Здесь $U_i(S_i)$ — множество допустимых управлений в состоянии S_i.

Поскольку все смены необходимо распределить полностью, граничное условие имеет вид

$$V_4(0) = 0, \qquad V_4(S) = -\infty \quad \text{при } S > 0.$$

Значение $-\infty$ обозначает недопустимое завершение, при котором линии закончились, а смены остались нераспределёнными.

Для восстановления оптимальных планов сохраняем все управления, на которых достигается максимум:

$$A_i(S) = \arg \max_{x_i \in U_i(S)} \left[w_i(x_i) + V_{i+1}(S - x_i) \right].$$

4 Ручное решение

Расчёт функций Беллмана выполняется в обратном порядке: от третьей линии к первой.

4.1 Третий этап

Третья линия является последней, поэтому она должна получить весь оставшийся ресурс:

$$x_3 = S_3.$$

Следовательно,

$$V_3(S_3) = w_3(S_3), \qquad A_3(S_3) = \{S_3\}.$$

S_3	1	2	3	4	5	6	7
$V_3(S_3)$	5	10	14	17	19	21	22
x_3^*	1	2	3	4	5	6	7

Например, если перед третьей линией осталось четыре смены, то

$$x_3 = 4, \qquad V_3(4) = w_3(4) = 17.$$

4.2 Второй этап

Для второго этапа

$$V_2(S_2) = \max_{1 \le x_2 \le S_2 - 1} \left[w_2(x_2) + V_3(S_2 - x_2) \right].$$

Одна смена обязательно оставляется третьей линии.

Рассмотрим подробно состояние $S_2 = 7$:

$$\begin{aligned}
x_2 = 1: &\quad 6 + V_3(6) = 6 + 21 = 27,\\
x_2 = 2: &\quad 11 + V_3(5) = 11 + 19 = 30,\\
x_2 = 3: &\quad 15 + V_3(4) = 15 + 17 = 32,\\
x_2 = 4: &\quad 18 + V_3(3) = 18 + 14 = 32,\\
x_2 = 5: &\quad 20 + V_3(2) = 20 + 10 = 30,\\
x_2 = 6: &\quad 22 + V_3(1) = 22 + 5 = 27.
\end{aligned}$$

Следовательно,

$$\boxed{V_2(7) = 32}, \qquad \boxed{A_2(7) = \{3, 4\}}.$$

Полная расчётная таблица второго этапа:

S_2	$x_2 = 1$	$x_2 = 2$	$x_2 = 3$	$x_2 = 4$	$x_2 = 5$	$x_2 = 6$	$x_2 = 7$	$V_2(S_2)$	$A_2(S_2)$
2	11	–	–	–	–	–	–	11	{1}
3	16	16	–	–	–	–	–	16	{1,2}
4	20	21	20	–	–	–	–	21	{2}
5	23	25	25	23	–	–	–	25	{2,3}
6	25	28	29	28	25	–	–	29	{3}
7	27	30	32	32	30	27	–	32	{3,4}
8	28	32	34	35	34	32	28	35	{4}

Знак “–” означает недопустимое управление: после выбора второй линии третьей не остаётся хотя бы одной смены.

4.3 Первый этап

Перед первой линией $S_1 = 12$. Она должна получить не менее 4 смен, а двум следующим линиям необходимо оставить хотя бы по одной, поэтому

$$x_1 \in \{4, 5, 6, 7, 8, 9, 10\}.$$

Используем

$$V_1(12) = \max_{x_1 = 4, \ldots, 10} \left[w_1(x_1) + V_2(12 - x_1) \right].$$

x_1	$w_1(x_1)$	$S_2 = 12 - x_1$	$V_2(S_2)$	сумма
4	18	8	35	**53**
5	21	7	32	**53**
6	23	6	29	52
7	25	5	25	50
8	26	4	21	47
9	27	3	16	43
10	28	2	11	39

Следовательно,

$$\boxed{V_1(12) = 53}, \qquad \boxed{A_1(12) = \{4, 5\}}.$$

4.4 Восстановление оптимальных планов

После обратного расчёта выполняется прямой проход из состояния $S_1 = 12$.

Если $x_1 = 4$, то

$$S_2 = 8, \qquad A_2(8) = \{4\}.$$

Поэтому $x_2 = 4$, а третьей линии остаётся $x_3 = 4$. Получаем

$$\boxed{(4, 4, 4)}.$$

Если $x_1 = 5$, то

$$S_2 = 7, \qquad A_2(7) = \{3, 4\}.$$

Поэтому возникают ещё два оптимальных плана:

$$\boxed{(5, 3, 4)}, \qquad \boxed{(5, 4, 3)}.$$

4.5 Проверка ограничений и целевой функции

Для первого плана:

$$4 + 4 + 4 = 12, \qquad W = 18 + 18 + 17 = 53.$$

Для второго:

$$5 + 3 + 4 = 12, \qquad W = 21 + 15 + 17 = 53.$$

Для третьего:

$$5 + 4 + 3 = 12, \qquad W = 21 + 18 + 14 = 53.$$

Во всех случаях первая линия получает не менее 4 смен, вторая и третья — не менее одной.

Итак,

$$\boxed{W^* = 53}$$

при трёх оптимальных распределениях

$$\boxed{(4, 4, 4), \quad (5, 3, 4), \quad (5, 4, 3)}.$$

5 Программная проверка

Программа реализует рекуррентную схему Беллмана. Исходные данные варианта задаются отдельно от алгоритма. На обратном проходе вычисляются значения функций Беллмана и сохраняются все управления, на которых достигается максимум. После этого прямым рекурсивным проходом восстанавливаются все оптимальные планы.

Код программы:

```
from math import inf


def solve_resource_dp(values, total_resource, allowed):
    n = len(values)

    value = [
        [-inf] * (total_resource + 1)
        for _ in range(n + 1)
    ]
```

```
    actions = [
        [[] for _ in range(total_resource + 1)]
        for _ in range(n)
    ]

    # Only zero remainder is valid after the last stage.
    value[n][0] = 0

    # Backward Bellman pass.
    for k in range(n - 1, -1, -1):
        for state in range(total_resource + 1):
            for x in sorted(allowed[k]):
                if x > state:
                    continue

                future = value[k + 1][state - x]
                if future == -inf:
                    continue

                candidate = values[k][x] + future

                if candidate > value[k][state]:
                    value[k][state] = candidate
                    actions[k][state] = [x]
                elif candidate == value[k][state]:
                    actions[k][state].append(x)

    optimum = value[0][total_resource]

    # Forward pass: restore all optimal plans.
    plans = []

    def restore(k, state, prefix):
        if k == n:
            if state == 0:
                plans.append(tuple(prefix))
            return

        for x in actions[k][state]:
            restore(k + 1, state - x, prefix + [x])

    restore(0, total_resource, [])

    return optimum, plans, value, actions


def main():
    values = [
        {1: 22, 2: 28, 3: 30, 4: 18, 5: 21, 6: 23, 7: 25, 8: 26, 9: 27, 10: 28, 11:
            29},
        {1: 6, 2: 11, 3: 15, 4: 18, 5: 20, 6: 22, 7: 23, 8: 24, 9: 25, 10: 25, 11:
            26},
        {1: 5, 2: 10, 3: 14, 4: 17, 5: 19, 6: 21, 7: 22, 8: 23, 9: 24, 10: 24, 11:
```

```
        25},
]

total_resource = 12

allowed = [
    set(range(4, 11)),
    set(range(1, 8)),
    set(range(1, 8)),
]

optimum, plans, value, actions = solve_resource_dp(
    values,
    total_resource,
    allowed,
)

print("Lab 4. Dynamic programming")
print("Variant 17: first line order")
print()

print("Stage 3:")
for state in range(1, 8):
    print(
        f"S3 = {state}: V3 = {value[2][state]}, "
        f"optimal x3 = {actions[2][state]}"
    )
print()

print("Stage 2:")
for state in range(2, 9):
    print(
        f"S2 = {state}: V2 = {value[1][state]}, "
        f"optimal x2 = {actions[1][state]}"
    )
print()

print("Stage 1, S1 = 12:")
for x1 in sorted(allowed[0]):
    tail = value[1][total_resource - x1]
    if tail == -inf:
        continue

    candidate = values[0][x1] + tail
    print(
        f"x1 = {x1}: w1 = {values[0][x1]}, "
        f"S2 = {total_resource - x1}, V2 = {tail}, total = {candidate}"
    )
print()

print("Optimal value:", optimum)
print("All optimal plans:")
for plan in plans:
```

```
        stage_values = [values[k][plan[k]] for k in range(len(values))]
        print(
            f" {plan}: {stage_values[0]} + {stage_values[1]} + "
            f"{stage_values[2]} = {sum(stage_values)}"
        )


if __name__ == "__main__":
    main()
```

Программа получает:

```
Третий этап:
S3 = 1: V3 = 5, оптимальное x3 = [1]
S3 = 2: V3 = 10, оптимальное x3 = [2]
S3 = 3: V3 = 14, оптимальное x3 = [3]
S3 = 4: V3 = 17, оптимальное x3 = [4]
S3 = 5: V3 = 19, оптимальное x3 = [5]
S3 = 6: V3 = 21, оптимальное x3 = [6]
S3 = 7: V3 = 22, оптимальное x3 = [7]

Второй этап:
S2 = 2: V2 = 11, оптимальные x2 = [1]
S2 = 3: V2 = 16, оптимальные x2 = [1, 2]
S2 = 4: V2 = 21, оптимальные x2 = [2]
S2 = 5: V2 = 25, оптимальные x2 = [2, 3]
S2 = 6: V2 = 29, оптимальные x2 = [3]
S2 = 7: V2 = 32, оптимальные x2 = [3, 4]
S2 = 8: V2 = 35, оптимальные x2 = [4]

Первый этап, S1 = 12:
x1 = 4: w1 = 18, S2 = 8, V2 = 35, сумма = 53
x1 = 5: w1 = 21, S2 = 7, V2 = 32, сумма = 53
x1 = 6: w1 = 23, S2 = 6, V2 = 29, сумма = 52
x1 = 7: w1 = 25, S2 = 5, V2 = 25, сумма = 50
x1 = 8: w1 = 26, S2 = 4, V2 = 21, сумма = 47
x1 = 9: w1 = 27, S2 = 3, V2 = 16, сумма = 43
x1 = 10: w1 = 28, S2 = 2, V2 = 11, сумма = 39

Оптимальное значение: 53
Все оптимальные планы:
  (4, 4, 4): 18 + 18 + 17 = 53
  (5, 3, 4): 21 + 15 + 17 = 53
  (5, 4, 3): 21 + 18 + 14 = 53

Проверка полным перебором: 53
Планы перебора: [(4, 4, 4), (5, 3, 4), (5, 4, 3)]
```

Рис. 1: Вывод программы

Показатель	Результат
Оптимальное значение	53
Оптимальный план 1	$(4, 4, 4)$
Оптимальный план 2	$(5, 3, 4)$
Оптимальный план 3	$(5, 4, 3)$

Программный результат полностью совпадает с ручным решением: получено то же оптимальное значение и те же три оптимальных плана.

6 Оценка сложности

Пусть n — число этапов, а B — общий целочисленный ресурс. Для каждого этапа рассматривается $O(B)$ состояний, а в каждом состоянии в худшем случае перебирается $O(B)$

управлений. Поэтому временная сложность реализации составляет

$$O(nB^2).$$

Таблица функций Беллмана занимает

$$O(nB)$$

памяти. Для данного варианта $n = 3$, $B = 12$, поэтому объём вычислений невелик.

7 Вывод

Методом динамического программирования решена задача распределения 12 смен между тремя производственными линиями. Были определены этапы, состояния, управления и переходы, построено рекуррентное соотношение Беллмана, выполнен обратный расчёт и восстановлены оптимальные планы.

Итоговый ответ:

1. **Оптимальное значение:**
$$\boxed{W^* = 53}.$$

2. **Оптимальные планы:**
$$\boxed{(4,4,4), \qquad (5,3,4), \qquad (5,4,3)}.$$

3. **Единственность решения:** оптимальный план не единственный. Существуют три различных распределения смен, обеспечивающих одинаковый максимальный выпуск $W^* = 53$.

4. **Проверка:**
$$4+4+4=12, \qquad 18+18+17=53,$$
$$5+3+4=12, \qquad 21+15+17=53,$$
$$5+4+3=12, \qquad 21+18+14=53.$$

   Во всех планах выполняются ограничения
$$x_1 \geq 4, \qquad x_2 \geq 1, \qquad x_3 \geq 1.$$

   Ручное и программное решения полностью совпадают.

Существенными особенностями варианта являются ограничение $x_1 \geq 4$ для первой линии и обязательное полное распределение всех 12 смен.